Università degli Studi di Salerno
Dipartimento di Informatica
Corso di Fondamenti di Intelligenza Artificiale

SyntheticSoul

Deep Reinforcement Learning Agents for the Mantis Lords Boss Fight in Hollow Knight

Students:

Lamberti Giovanni
mat.

Plumitallo Antonio
mat.

Senatore Francesco
mat.

Ricciardelli Domenico
mat. 0512119238

Professor:

Prof. Fabio Palomba

Academic Year 2025/2026

GitHub Repository: https://github.com/jlgionny/SyntheticSoul

Contents

1 Introduction and Problem Definition **3**
1.1 Introduction . . . 3
1.2 PEAS Specification . . . 3
1.3 Environment Characteristics . . . 4
1.4 Technical Challenges . . . 4

2 System Architecture **5**
2.1 Architectural Overview . . . 5
2.2 Communication Protocol . . . 5
2.3 Game State Extraction (GameStateExtractor) . . . 6
2.4 Attack Pattern Detection (MantisAttackDetector) . . . 7
2.5 Action Execution (ActionExecutor) . . . 7
2.6 Victory Management and Episode Lifecycle . . . 8

3 State Preprocessing **9**
3.1 Version V1 — 34 Features . . . 9
3.2 Version V2 — 51 Features . . . 9
3.3 Frame Stacking . . . 9

4 AI Solution: Deep Reinforcement Learning **11**
4.1 Action Space . . . 11
4.2 Experience Management (Dynamic Dataset) . . . 11
4.2.1 Transition Structure . . . 11
4.2.2 Replay Buffer (DQN) . . . 11
4.3 Deep Q-Network (DQN) . . . 12
4.3.1 Architecture: Dueling DQN . . . 12
4.3.2 DQN Training Mechanisms . . . 13
4.4 Proximal Policy Optimization (PPO) . . . 13
4.4.1 Actor-Critic Architecture with LSTM . . . 13
4.4.2 Clipped Objective Function . . . 14
4.4.3 Advanced PPO Mechanisms . . . 14
4.5 DQN vs PPO Comparison . . . 15

5 Experiments: 5-Phase Curriculum Learning **16**
5.1 Phase Pipeline . . . 16
5.2 Reward Function Details . . . 16
5.3 Pattern Reward Bonus . . . 17

6 Multi-Instance Training **19**
6.1 Distributed Training Architecture . . . 19
6.2 Hall of Fame . . . 19
6.3 Per-Phase Hyperparameter Configuration . . . 19

7 Implementation Details **21**

7.1 Numerical Stability 21
7.2 Technology Stack 21
7.3 Project Structure 21

8 Conclusions and Future Work **23**
8.1 Main Contributions 23
8.2 Future Work 23

Chapter 1

Introduction and Problem Definition

1.1 Introduction

The **SyntheticSoul** project explores the application of Deep Reinforcement Learning (DRL) in complex, non-deterministic game environments. Unlike classical AI domains such as chess or Go, where the state space is discrete and fully observable, *Hollow Knight* features a continuous state space, real-time dynamics and adversaries with stochastic attack patterns.

Hollow Knight is a 2D action-platformer developed by Team Cherry, renowned for its precise and challenging combat system. The **Mantis Lords** represent one of the game's most iconic boss fights: a multi-phase encounter against agile enemies that attack with horizontal dashes, diagonal lunges, boomerang projectile throws and vertical plunges. The second phase introduces *two simultaneous mantises*, demanding heightened spatial awareness and the ability to manage multiple threats concurrently.

The main goal of the project is to train autonomous agents capable of defeating the Mantis Lords in the **Hall of Gods** (the game's boss-rush mode), comparing the performance of two fundamental Deep RL approaches:

1. **DQN** (Deep Q-Network) — a *value-based, off-policy* method that estimates the value of each action through a Dueling neural network;
2. **PPO** (Proximal Policy Optimization) — a *policy-based, on-policy* method that directly learns a stochastic policy, augmented with LSTM temporal memory.

1.2 PEAS Specification

The agent's operating environment is defined according to the PEAS framework:

- **Performance:** Survival time (episode duration), damage dealt to the boss, number of mantises killed, final victory (all three mantises defeated).
- **Environment:** Mantis Lords arena in the Hall of Gods (Godhome) — a rectangular platform with lateral spikes and a multi-phase boss.
- **Actuators:** Virtual controller with 8 discrete actions: movement (left, right, up, down), jump, attack, dash and spell.
- **Sensors:** Direct extraction from game memory (HP, position, velocity of player and boss), terrain raycasts, PlayMaker FSM readings for attack patterns, projectile detection.

1.3 Environment Characteristics

- **Single agent:** only one agent (the knight) must defeat the boss.
- **Partially observable:** the agent perceives only the information extracted by the mod, not the entire visual scene.
- **Stochastic:** the boss selects attack patterns pseudo-randomly via PlayMaker FSMs.
- **Dynamic:** the game advances in real time regardless of the agent's decisions.
- **Continuous:** positions, velocities and distances are floating-point values.
- **Episodic:** each attempt (from scene load to death or victory) constitutes an independent episode.

1.4 Technical Challenges

The project tackles several non-trivial challenges:

- **Game interfacing:** Hollow Knight exposes no API for external agents; a complete Unity/C# mod had to be developed.
- **Real-time synchronization:** the game–agent loop must operate at 20 Hz without desynchronization.
- **Heterogeneous observation space:** the state must be extracted from diverse Unity components (Rigidbody2D, PlayMaker FSM, HealthManager, etc.).
- **Multi-phase boss:** the Mantis Lords change behaviour after the first kill, switching from a single opponent to two simultaneous attackers.
- **Reward shaping:** designing reward functions that progressively guide learning without introducing degenerate behaviours.

Chapter 2

System Architecture

The SyntheticSoul system comprises two macro-components: a **C# mod** running inside the Hollow Knight process (Unity side) and a **Python training framework** executing the RL algorithms. Communication takes place over TCP sockets on localhost.

2.1 Architectural Overview

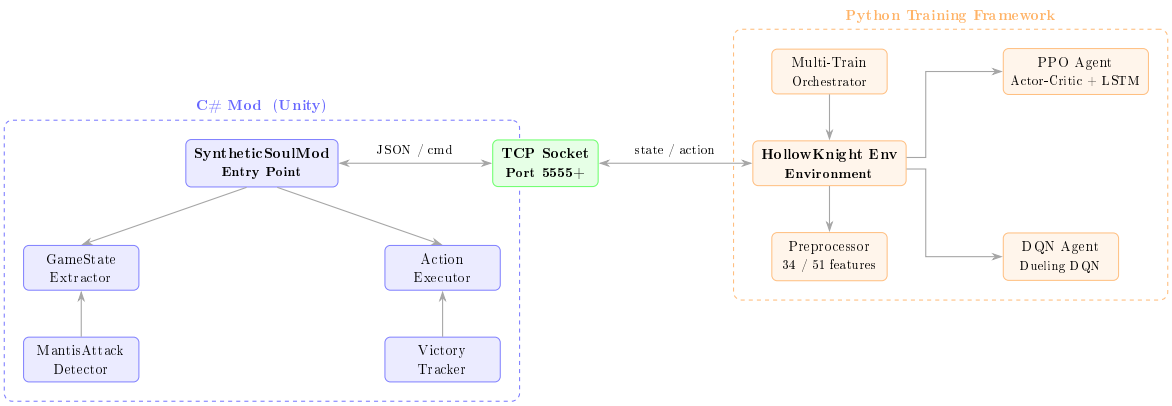

Figure 2.1: System architecture: communication between the C# Mod and the Python Training Framework via TCP sockets.

2.2 Communication Protocol

Communication between the mod and the agent follows a synchronous *request–response* protocol:

1. The mod extracts the game state and serializes it as **JSON**.
2. The JSON string is sent to the Python client via TCP socket (newline-delimited).
3. Python receives the state, preprocesses it, selects an action and sends back a command string (e.g. `MOVE_LEFT`, `ATTACK`, `DASH`).
4. The mod receives the command and executes it through the virtual input device.

This loop operates at approximately **20 Hz** (`UPDATE_INTERVAL = 0.05 s`). Socket reads on the mod side are **blocking**: the game waits for Python's response before proceeding, ensuring perfect synchronization at the cost of inference speed. The `SocketCommunicator` component manages persistent connections with automatic reconnection, command queues for burst handling and configurable timeouts (10 s).

```
// The read BLOCKS the thread until Python responds.
// This forces the game to run at Python's speed.
byte[] buffer = new byte[4096];
int bytesRead = stream.Read(buffer, 0, buffer.Length);

if (bytesRead > 0)
{
    string rawData = Encoding.UTF8.GetString(buffer, 0, bytesRead);
    string[] commands = rawData.Split(
        new char[] { '\n', '\r' },
        StringSplitOptions.RemoveEmptyEntries);
    foreach (string cmd in commands)
        commandQueue.Enqueue(cmd.Trim().ToUpper());
}
```

Listing 2.1: Blocking socket read (SocketCommunicator.cs)

2.3 Game State Extraction (GameStateExtractor)

The class `GameStateExtractor.cs` extracts the game state at every step. The `GameState` structure contains roughly 80 fields organized into categories:

Table 2.1: Game state categories extracted by the mod.

Category	**Key Fields**
Player	Position (x, y), velocity (v_x, v_y), HP, Soul, isGrounded, canDash, canAttack
Boss (threat-locked)	Position, velocity, HP, relative distance and position to player, isFacingBoss
Boss Intent	Current action with priority: Drop > Dash > Throw > Idle
Hazards	Nearby projectiles with relative position, velocity, distance
Attack Patterns	Primary/secondary mantis pattern (8 patterns + wind-up/active/recovery phases)
Terrain	5 raycast values detecting ground below and beside the player
Victory Tracker	Session wins, current streak, win rate

A critical aspect is the **threat locking** mechanism: boss-related fields always point to the most dangerous mantis — the one in an active attack phase closest to the player — rather than simply the nearest one by distance.

```
private void ExtractTerrainInfo(HeroController hero, GameState
    state)
{
    Vector2 rayOrigin = new Vector2(playerPos.x, playerPos.y + 0.5f
    );
    // Downward raycast
    RaycastHit2D groundHit = Physics2D.Raycast(
        rayOrigin, Vector2.down, 5f, terrainLayer);
    // Normalization (0-1)
    state.terrainInfo[0] = (groundHit.collider != null)
```

```
        ? Mathf.Clamp01(groundHit.distance / 3f) : 1.0f;
}
```

Listing 2.2: Terrain extraction via Raycast (GameStateExtractor.cs)

2.4 Attack Pattern Detection (MantisAttackDetector)

The attack-pattern detector is one of the project's most significant contributions. It combines three information sources to classify each mantis's behaviour in real time:

1. **PlayMaker FSM reading:** each mantis has an FSM named "Mantis Lord" whose states correspond to different attacks. The detector reads the active state and classifies it via substring matching.
2. **Kinematic analysis:** the mantis's velocity (from Rigidbody2D or derived from position deltas) confirms or disambiguates the pattern.
3. **Temporal tracking:** the duration of each state is tracked to identify wind-up, active attack and recovery phases.

Table 2.2: Mantis Lords attack patterns recognized by the detector.

ID	Pattern	Description & Kinematic Thresholds
0	IDLE	No attack in progress
1	DASH_HORIZ	Horizontal ground lunge ($\lvert v_x\rvert > 5$, $\lvert v_y\rvert < 3$)
2	DASH_DIAG	Diagonal dash from above ($\lvert v_x\rvert > 3$ and $\lvert v_y\rvert > 3$)
3	DAGGER_THROW	Boomerang sickle throw from wall
4	WALL_THROW	Attack from lateral wall
5	LAND_ATTACK	Vertical plunge from above ($v_y < -5$)
6	WIND_UP	Attack preparation (reaction window)
7	RECOVERING	Post-attack cooldown (punishment window)

Classification follows a hybrid approach: the FSM state is checked first, then confirmed with kinematic thresholds. A purely kinematic fallback (speed > 15 units/s) handles cases where the FSM is unreadable.

2.5 Action Execution (ActionExecutor)

Action execution happens through a **virtual input device** registered in Unity's InControl system. The class `SyntheticSoulInputDevice` simulates a virtual controller with 8 buttons whose bindings are injected into the game's `HeroActions` via *reflection*.

```
private void BindAction(Type type, string name, InputControl
    control)
{
    var field = type.GetField(name,
        BindingFlags.NonPublic | BindingFlags.Instance |
    BindingFlags.Public);
    if (field != null)
    {
```

```
        var action = field.GetValue(heroActions) as PlayerAction;
        if (action != null && control != null)
            action.AddBinding(new DeviceBindingSource(control.
    Target));
     }
}
```

Listing 2.3: Binding via Reflection (ActionExecutor.cs)

Each action has a configured duration: directional movements persist for a defined interval, while attack, dash and jump are released automatically after a short tap. Before execution, the system verifies private `HeroController` methods (`CanDash()`, `CanAttack()`, `CanJump()`) via reflection to avoid invalid inputs. The system also supports composite actions such as `JUMP_ATTACK` and `DASH_ATTACK`, essential for advanced combat.

2.6 Victory Management and Episode Lifecycle

The `VictoryTracker` manages the victory sequence and maintains session-level training statistics. In the Hall of Gods the Mantis Lords fight requires killing all three mantises; the tracker confirms victory only when `mantisLordsKilled >= 3`.

The episode lifecycle is managed entirely by the mod:

1. **Scene Load:** the scene `GG_Mantis_Lords` is loaded; the mod resets all counters, trackers and detectors.
2. **Hero Restoration:** after loading, the mod waits for the boss intro, restores player HP/Soul and removes residual death objects.
3. **Training Loop:** every 50 ms the mod extracts the state, sends it to Python and waits for the action.
4. **Termination:** upon death or victory the mod sends the final state with the `isDead`/`bossDefeated` flags and automatically reloads the scene.

Chapter 3

State Preprocessing

The raw JSON state received from the mod is transformed into a normalized numerical vector by `preprocess.py`. All values are normalized to $[-1, +1]$ or $[0, 1]$ to stabilize training. Two preprocessing versions are available.

3.1 Version V1 — 34 Features

The base version extracts 34 features organized into 7 groups:

Table 3.1: State vector composition — V1 (34 features).

Group	#	**Detail**
Player Status	5	HP/20, Soul/100, canDash, canAttack, isGrounded
Player Velocity	2	$v_x/20$, $v_y/20$ clamped to $[-1, 1]$
Terrain	5	5 normalized raycast values $[0, 1]$
Boss Position	4	relativeX, relativeY, distance/50, isFacingBoss
Boss Status	4	$v_x/20$, $v_y/20$, HP/100, mantisKilled/3
Boss Intent	4	One-hot encoding (4 classes: Idle/Dash/Drop/Throw)
Hazards	10	2×5 features (relX, relY, v_x, v_y, dist)

3.2 Version V2 — 51 Features

Version V2 extends V1 with 17 additional features from the `MantisAttackDetector`:

Table 3.2: Additional features in V2 (+17 over V1).

Group	#	**Detail**
Primary Pattern	8	One-hot of primary pattern (8 classes, see Tab. 2.2)
Temporal State	3	isWindingUp, isActive, isRecovering
Primary Velocity	2	$v_x/25$, $v_y/25$ of the primary mantis
Secondary Mantis	2	Secondary pattern/7, isActive secondary
Combat Info	2	activeMantisCount/3, anyMantisAttacking

3.3 Frame Stacking

To provide temporal information to the agents (especially DQN, which lacks an LSTM), the system uses **frame stacking**: the last $N = 4$ frames are concatenated, yielding an input dimension of:

$$d_{\text{input}} = d_{\text{features}} \times N = 51 \times 4 = 204 \quad \text{(V2)} \tag{3.1}$$

```python
class FrameStacker:
    """Stack of N consecutive frames. State [F] -> [F x N]."""
    def __init__(self, stack_size, state_dim):
        self.frames = deque(maxlen=stack_size)

    def reset(self, initial_state):
        self.frames.clear()
        for _ in range(self.stack_size):
            self.frames.append(initial_state)
        return np.concatenate(list(self.frames))

    def step(self, state):
        self.frames.append(state)
        return np.concatenate(list(self.frames))
```

Listing 3.1: Frame stacker implementation (multi_train.py)

Chapter 4

AI Solution: Deep Reinforcement Learning

4.1 Action Space

Both agents share a discrete action space with 8 actions:

Table 4.1: Agent action space.

ID	Action	In-Game Effect
0	MOVE_LEFT	Move left
1	MOVE_RIGHT	Move right
2	UP	Look up
3	DOWN	Look down
4	JUMP	Jump (or double jump)
5	ATTACK	Nail strike
6	DASH	Quick dash
7	SPELL	Cast spell (Vengeful Spirit, etc.)

4.2 Experience Management (Dynamic Dataset)

Unlike classical supervised learning where the dataset is static, in SyntheticSoul the dataset is **generated dynamically** by the agent during interaction with the environment.

4.2.1 Transition Structure

Each interaction is stored as a tuple e_t:

$$e_t = (s_t,\ a_t,\ r_t,\ s_{t+1},\ d_t) \tag{4.1}$$

where s_t is the current state (204-dimensional vector with V2 frame stacking), a_t the executed action (integer 0–7), r_t the reward, s_{t+1} the next state and d_t the termination flag.

4.2.2 Replay Buffer (DQN)

For DQN we implemented a circular FIFO buffer with capacity $N = 100{,}000$. This allows the agent to discard experiences generated by immature policies while maintaining sufficient variance to avoid overfitting.

```
class ReplayBuffer:
    def __init__(self, capacity=100000):
```

```
        self.memory = deque(maxlen=capacity)  # auto FIFO

    def push(self, state, action, reward, next_state, done):
        self.memory.append(Transition(state, action, reward,
    next_state, done))

    def sample(self, batch_size):
        # Random sampling to break temporal correlation
        return random.sample(self.memory, batch_size)
```

Listing 4.1: Replay buffer implementation (dqn_agent.py)

For the PPO agent the dataset is *ephemeral*: it is accumulated over a full episode, used to compute gradients (4 epochs of mini-batches of 128) and discarded immediately after the update, ensuring strictly *on-policy* learning.

4.3 Deep Q-Network (DQN)

4.3.1 Architecture: Dueling DQN

As implemented in `dqn_net.py`, the agent uses a **Dueling DQN** architecture [4] rather than a plain feed-forward network. This architecture separates the estimation of the state value $V(s)$ from the advantage of each action $A(s, a)$. This is especially useful in Hollow Knight because in many frames (e.g. while walking away from the boss) the chosen action is irrelevant to the state's value.

The final Q-value is reconstructed as:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right) \tag{4.2}$$

where θ are the shared parameters, α the Advantage stream parameters and β the Value stream parameters.

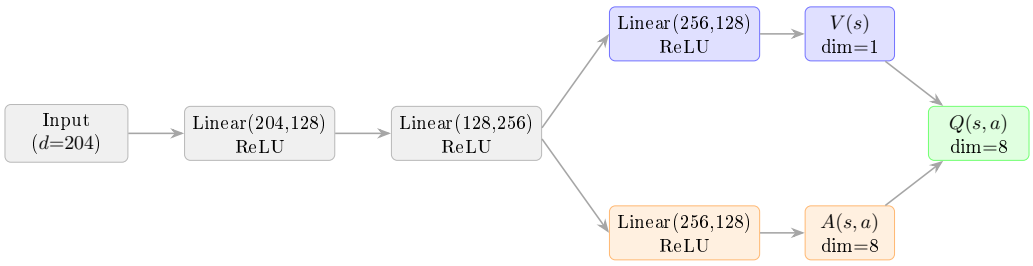

Figure 4.1: Dueling DQN architecture. Shared layers extract features; two separate streams estimate $V(s)$ and $A(s, a)$.

```
def forward(self, state):
    # Shared layers
    x = F.relu(self.fc1(state))
    x = F.relu(self.fc2(x))
    # Value stream
    value = F.relu(self.value_fc(x))
    value = self.value_out(value)            # [batch, 1]
    # Advantage stream
```

```
    advantage = F.relu(self.advantage_fc(x))
    advantage = self.advantage_out(advantage) # [batch, 8]
    # Q = V + (A - mean(A))
    q_values = value + (advantage - advantage.mean(dim=1, keepdim=True))
    return q_values
```

Listing 4.2: Dueling DQN forward pass (dqn_net.py)

4.3.2 DQN Training Mechanisms

The DQN agent implements several improvements over vanilla DQN:

- **Double DQN:** the policy network selects the best action in the next state, but the target network estimates its value, reducing overestimation:

$$y = r + \gamma \cdot Q_{\theta^-}\left(s', \arg\max_{a'} Q_\theta(s', a')\right) \tag{4.3}$$

- **Experience Replay:** circular buffer of 100,000 transitions, sampled in mini-batches of 256.
- **Soft Target Update** with $\tau = 0.005$:

$$\theta^- \leftarrow \tau \cdot \theta + (1 - \tau) \cdot \theta^- \tag{4.4}$$

- **Smooth L1 Loss** (Huber Loss) instead of MSE for improved stability.
- **Gradient Clipping** with $\|\nabla\|_{\max} = 1.0$.
- **Epsilon-greedy** with exponential decay:

$$\epsilon = \epsilon_{\text{end}} + (\epsilon_{\text{start}} - \epsilon_{\text{end}}) \cdot e^{-\text{steps}/\text{decay}} \tag{4.5}$$

4.4 Proximal Policy Optimization (PPO)

While DQN estimates action values (value-based), PPO directly learns the policy $\pi_\theta(a|s)$ (policy-based). We chose PPO [2] for its stability and ability to converge in complex environments.

4.4.1 Actor-Critic Architecture with LSTM

A major challenge in Hollow Knight is partial non-Markovianity: understanding the trajectory of a boomerang projectile requires knowledge of previous frames. For this reason we integrated an **LSTM** layer that maintains a hidden state across episode steps. The architecture is:

- **Feature Extractor:** Linear(d_{in}, 384) → ReLU → LayerNorm → Linear(384, 384) → ReLU → LayerNorm
- **LSTM:** 1 layer, hidden size 384
- **Actor Head** (π_θ): Linear(384, 128) → ReLU → Linear(128, 8)
- **Critic Head** (V_ϕ): Linear(384, 128) → ReLU → Linear(128, 1)

Weights are initialized with **orthogonal initialization** (gain $\sqrt{2}$), and NaN/Inf protections are applied to logits (clamping to $[-20, 20]$).

```
def forward(self, x):
    features = self.feature_extractor(x)
    if self.use_lstm:
        if len(features.shape) == 2:
            features = features.unsqueeze(1)  # [batch, 1, hidden]
        if self.hidden_state is not None:
            if self.hidden_state[0].shape[1] != features.shape[0]:
                self.hidden_state = None
        features, self.hidden_state = self.lstm(features, self.
    hidden_state)
        # CRITICAL: detach to prevent backprop through entire
    episode
        self.hidden_state = (
            self.hidden_state[0].detach(),
            self.hidden_state[1].detach())
        features = features.squeeze(1)
    action_logits = self.actor(features)
    value = self.critic(features)
    return action_logits, value
```

Listing 4.3: Forward pass with LSTM (ppo_agent.py)

4.4.2 Clipped Objective Function

The core of PPO is the clipped objective that prevents excessively large policy updates:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \Big) \right] \tag{4.6}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t) \,/\, \pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the advantage estimated via **GAE** (Generalized Advantage Estimation):

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \qquad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{4.7}$$

The total loss combines three components:

$$L = L^{\text{CLIP}} + c_1 \cdot L^{\text{VF}} - c_2 \cdot H[\pi_\theta] \tag{4.8}$$

with $c_1 = 0.5$ (value loss coefficient), c_2 decaying over time (entropy for exploration) and L^{VF} computed with Smooth L1 Loss. Updates are performed over 4 epochs on mini-batches of 128.

4.4.3 Advanced PPO Mechanisms

- **Learning Rate Scheduling:** linear decay from lr_{start} to $\text{lr}_{\text{start}} \times$ lr_end_factor.
- **Dynamic Entropy:** the coefficient c_2 follows a cosine annealing schedule:

$$c_2(t) = c_{2,\text{end}} + (c_{2,\text{start}} - c_{2,\text{end}}) \cdot \frac{1 + \cos(\pi \cdot t/T)}{2} \tag{4.9}$$

- **Batch Size Safety:** automatic batch dimension check for the LSTM hidden state, with reset if the dimension changes.
- **Hidden State Detach:** the LSTM hidden state is detached at every step to prevent backpropagation through the entire episode.

4.5 DQN vs PPO Comparison

Table 4.2: Comparison of the two agents' key characteristics.

Aspect	**DQN**	**PPO**
Paradigm	Value-based, off-policy	Policy-based, on-policy
Network	Dueling DQN (128-256-128)	Actor-Critic + LSTM (384)
Exploration	Decaying ϵ-greedy	Policy entropy
Temporal memory	Frame stacking (4×)	Built-in LSTM
Replay Buffer	Yes (100k transitions)	No (on-policy, per-episode)
Target Network	Yes (soft update τ=0.005)	No
Update timing	Every step (if buffer ready)	End of episode (4 epochs)
Parameters	∼117k	∼580k

Chapter 5

Experiments: 5-Phase Curriculum Learning

Hollow Knight's intrinsic difficulty is extremely high: if the agent tries to attack immediately it dies within seconds without receiving meaningful feedback. To mitigate the *sparse reward* problem and accelerate convergence, we adopted a **5-phase Curriculum Learning** approach.

5.1 Phase Pipeline

Figure 5.1: Curriculum learning pipeline with automatic promotion conditions.

5.2 Reward Function Details

Each phase has a dedicated reward function implemented in `hollow_knight_env.py`. All rewards are scaled by 1/5 to stabilize training.

Phase 1: Survival

Goal: learn to move, use dash and jump to dodge attacks.

Living reward (per step)	+0.01
Damage taken	−4.0
Spike damage (additional)	−4.0
Successful dodge (boss attacks, no hit)	+0.15
Wind-up reaction (jump/dash)	+0.05
Death	−3.0
Survival milestone (every 200 steps)	+0.5

Promotion: average steps ≥ 700 over the last 20 episodes.

Phase 2: First Hit

Goal: learn to strike the boss during recovery windows.

Living reward	$+0.005$
Damage dealt to boss	$+\Delta\text{HP} \times 0.15$
Hit during recovery (bonus)	$+0.3$
Attack during wind-up (penalty)	-0.1
Damage taken	-3.0
Death	-3.0

Promotion: average damage dealt ≥ 200 over the last 25 episodes.

Phase 3: Aggression

Goal: kill the first mantis. Maximize DPS.

Damage dealt	$+\Delta\text{HP} \times 0.2$
Boss HP threshold $< 200/100/50$	$+3.0/+5.0/+8.0$
Mantis kill	$+40.0$
Full victory	$+50.0$
Damage taken	-1.5
Death	-5.0

Promotion: average kills ≥ 0.5 over the last 30 episodes.

Phase 4: Dual Mantis

Goal: manage two simultaneous mantises (boss fight phase 2).

Similar to Phase 3 with increased damage penalty (-3.5), progressive kill bonuses ($+40 \times n$), and bonuses for dodging simultaneous attacks ($+0.08$).

Promotion: average kills ≥ 2.0 over the last 30 episodes.

Phase 5: Mastery

Goal: complete victory, optimize time and damage taken.

Damage dealt	$+\Delta\text{HP} \times 0.25$
Kill	$+40.0$
Victory	$+50.0$
Time bonus: $\max(0, (3000 - \text{steps})/100)$	variable
No-hit victory (additional)	$+30.0$
Damage taken	-5.0
Death	-8.0

Promotion: win rate $\geq 50\%$ over the last 50 episodes.

5.3 Pattern Reward Bonus

In addition to the phase reward, a bonus based on attack patterns is computed by `preprocess.py`:

Table 5.1: Reward bonus based on the mantis's attack phase.

Mantis Phase	**Agent Action**	**Bonus**
Wind-up	Dodge/Move (actions 0–4, 6)	+0.02
Wind-up	Attack (action 5)	−0.01
Active attack	Dodge (actions 4, 6)	+0.05
Recovery	Attack (actions 5, 7)	+0.08
Secondary active	Dodge (actions 4, 6)	+0.03

Chapter 6

Multi-Instance Training

6.1 Distributed Training Architecture

The system supports parallel training on **multiple game instances**, each connected to a separate agent instance on a different TCP port. The orchestrator `multi_train.py` manages:

- Launch of N parallel processes (via `multiprocessing` with `spawn` for CUDA compatibility).
- Sharing of best models through a file-locked **Hall of Fame**.
- Periodic synchronization: under-performing instances load models from the Hall of Fame.
- Sequential pipeline through all 5 phases, with automatic best-model transfer to the next phase.

6.2 Hall of Fame

The Hall of Fame maintains the **Top-K** models ($K = 3$) shared among all instances:

- Each instance may occupy at most **one slot** (to ensure diversity).
- Updating a slot requires a minimum improvement of +0.5 reward.
- Ranking is by descending reward; weaker models are discarded.
- Access is protected by **file locking** (`filelock`) for multi-process safety.

Periodically (every `sync_interval` $\times 3$ episodes), instances whose average performance falls below 30% of the global best load a random model from the Hall of Fame.

6.3 Per-Phase Hyperparameter Configuration

Table 6.1: Key hyperparameters for each training phase.

Parameter	**P1**	**P2**	**P3**	**P4**	**P5**
Episodes	400	600	600	700	1000
γ	0.99	0.995	0.995	0.995	0.998
PPO LR	$3{\cdot}10^{-4}$	$2{\cdot}10^{-4}$	$1.5{\cdot}10^{-4}$	10^{-4}	$5{\cdot}10^{-5}$
Entropy	.08→.03	.06→.02	.05→.015	.04→.01	.03→.005
DQN LR	10^{-4}	$5{\cdot}10^{-5}$	$3{\cdot}10^{-5}$	$2{\cdot}10^{-5}$	10^{-5}
ϵ	.60→.10	.40→.08	.30→.05	.25→.05	.15→.03

The general trend is to reduce exploration and learning rates in later phases while increasing the discount factor γ to favour long-term strategies.

Chapter 7

Implementation Details

7.1 Numerical Stability

Several measures were adopted to guarantee numerical stability:

- **LayerNorm** in the PPO feature extractor to normalize activations.
- **Gradient clipping:** max norm 0.5 (PPO) and 1.0 (DQN).
- **NaN protection:** Actor logits are checked and zeroed if they contain NaN/Inf.
- **Advantage normalization:** skipped if the buffer contains fewer than 2 samples (to avoid `std()` on a single element).
- **Smooth L1 Loss** (Huber) for both agents, more robust to outliers than MSE.
- **Xavier uniform** (DQN) and **orthogonal** (PPO) weight initialization.

7.2 Technology Stack

Table 7.1: Technologies used in the project.

Component	Technology
Game Mod	C# / Unity, Hollow Knight Modding API, InControl, PlayMaker
Communication	TCP Sockets, JSON (Newtonsoft.Json), blocking reads
RL Framework	Python 3, PyTorch (with CUDA support)
Parallelism	Python multiprocessing (spawn), filelock for synchronization
Logging	Shared CSV (file-locked), custom DesktopLogger to file

7.3 Project Structure

```
SyntheticSoul/
+-- mod/                          # C# Mod (Unity)
|   +-- SyntheticSoulMod.cs       # Entry point, lifecycle
|   +-- GameStateExtractor.cs     # Game state extraction
|   +-- ActionExecutor.cs         # Virtual input device
|   +-- SocketCommunicator.cs     # TCP communication
|   +-- MantisAttackDetector.cs   # Pattern detection
|   +-- VictoryTracker.cs         # Victory tracking
|   +-- DesktopLogger.cs          # Debug logging
+-- src/
    +-- agents/
    |   +-- ppo_agent.py          # PPO with LSTM
```

```
|   +-- dqn_agent.py            # DQN with Dueling
+-- models/
|   +-- dqn_net.py              # DQN/Dueling architectures
|   +-- actor_critic.py         # Actor-Critic network
+-- env/
|   +-- hollow_knight_env.py    # Environment wrapper
|   +-- preprocess.py           # Preprocessing V1/V2
+-- training/
    +-- multi_train.py          # Multi-instance orchestrator
```

Chapter 8

Conclusions and Future Work

The SyntheticSoul project demonstrates the feasibility of applying Deep Reinforcement Learning techniques to a complex boss fight in a commercial game, addressing significant challenges in real-time interfacing, synchronization and reward engineering.

8.1 Main Contributions

- A **complete framework** for training RL agents on Hollow Knight, reusable for other boss fights.
- An **attack pattern detector** based on the combination of FSM reading, kinematic analysis and temporal tracking.
- A **5-phase curriculum learning** system with automatic promotion based on performance metrics.
- A **multi-instance training** infrastructure with a shared Hall of Fame to accelerate convergence.
- A **direct comparison** between DQN (value-based) and PPO (policy-based) on the same task, highlighting the trade-offs between the two approaches.

8.2 Future Work

- Extension to other Hollow Knight bosses (Hornet, Soul Master, Nightmare King Grimm).
- Integration of **visual observations** (pixels) via CNN, eliminating the dependency on direct game-state reading.
- Implementation of **self-play** or **population-based training** for greater strategy diversification.
- Use of **imitation learning** from human replays as pre-training to accelerate initial learning.
- Training speed optimization through game acceleration (`TimeScale > 1`).

Bibliography

[1] Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

[2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347.

[3] Mnih, V., et al. (2015). *Human-level control through deep reinforcement learning*. Nature, 518(7540), 529–533.

[4] Wang, Z., Schaul, T., Hessel, M., et al. (2016). *Dueling Network Architectures for Deep Reinforcement Learning*. Proceedings of ICML.

[5] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780.

[6] Hollow Knight Modding API. https://hk-modding.github.io/api/